

GeoLog — Persistência poliglota

Persistência Poliglota para Monitoramento Geoespacial e Logística

Instituição: UNIPÊ

Disciplina: implementação e Ger. de Bancos de Dados Nosql

Atividade: Desafio Integrador GeoLog

Professor: Ricardo Roberto

Grupo: Wellingson Sousa da Silva

Data: 29/09/2026

1. Contexto e Objetivo

A transportadora LogiTech Express opera uma frota que gera, simultaneamente, dois perfis de dados muito distintos: registros cadastrais e transacionais de motoristas e veículos, que exigem integridade referencial e consultas relacionais; e telemetria de alta frequência (coordenadas GPS, temperatura, velocidade), com estrutura flexível e forte componente espacial. O objetivo deste trabalho é projetar e implementar a plataforma **GeoLog**, um dashboard unificado em Streamlit que aplica o padrão de **Persistência Poliglota**, integrando um banco relacional (SQLite) e um banco NoSQL orientado a documentos (MongoDB) para atender esse cenário de forma eficiente.

2. Padrão Arquitetural: Persistência Poliglota

Persistência Poliglota é o padrão em que uma mesma aplicação utiliza diferentes tecnologias de armazenamento, cada uma escolhida por ser a mais adequada ao tipo de dado e de consulta que precisa responder. Neste projeto:

- **SQLite** armazena os dados relacionais (motoristas, veículos), priorizando a integridade referencial (chave estrangeira `motorista_id`) e a simplicidade transacional.
- **MongoDB** armazena a telemetria em documentos GeoJSON, aproveitando índices `2dsphere` nativos para consultas geoespaciais de proximidade — algo que um SQL tradicional resolveria de forma bem mais custosa.

A aplicação Streamlit atua como camada de orquestração: consulta os dois bancos de forma independente e combina os resultados apenas em memória (com pandas), no momento da exibição — sem replicar dados entre os motores nem misturar suas responsabilidades.

3. Diagrama de Arquitetura

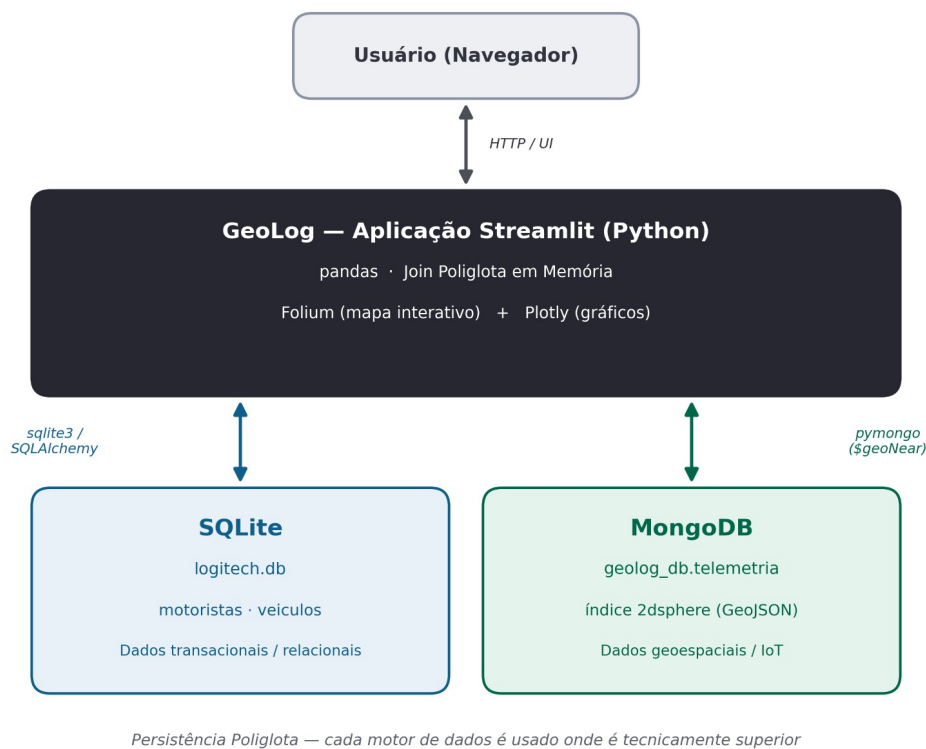


Figura 1 — Arquitetura poliglota do GeoLog: a aplicação Streamlit consome, de forma independente, o SQLite (dados relacionais) e o MongoDB (dados geoespaciais), combinando os resultados apenas na camada de apresentação.

4. Descrição dos Módulos

Módulo	Descrição
1 · Persistência & Seed	Inicializa as conexões, cria as tabelas <code>motoristas</code> e <code>veiculos</code> no SQLite, cria a coleção <code>telemetry</code> no MongoDB com índice <code>2dsphere</code> e popula ambos os bancos com os dados de teste.
2 · Geoprocessamento (Raio)	Permite escolher um ponto de apoio e um raio em km; executa uma agregação <code>\$geoNear</code> no MongoDB para localizar e ordenar por distância os veículos dentro do raio, exibidos em mapa Folium.
3 · Visão Unificada (Join)	Busca motoristas/veículos no SQLite e a última telemetria de cada veículo no MongoDB (agregação <code>\$group</code>), combinando os dois resultados em memória com <code>pandas.merge()</code> .
4 · Dashboard Analítico	KPIs (frota ativa, temperatura média, alertas de velocidade acima de 80 km/h) e gráficos Plotly: histórico de temperatura por veículo e distribuição de status dos motoristas.
Bônus · Simulador em Tempo Real	Botão que gera novas leituras de GPS/sensor com variação aleatória e insere no MongoDB, atualizando mapa e dashboard sem reiniciar a aplicação.

5. Decisões Técnicas de Implementação

- O índice `2dsphere` é criado programaticamente (`pymongo.GEOSPHERE`) na inicialização da aplicação, conforme exigido pelo enunciado, garantindo que as consultas geoespaciais funcionem sem intervenção manual no banco.
- Optou-se por `$geoNear` (estágio de agregação) em vez do operador simples `$near`, pois ele devolve a distância calculada (`distanceField`) diretamente no pipeline, permitindo ordenar e exibir a distância de cada veículo sem cálculo adicional em Python.
- O join poliglota é resolvido inteiramente em memória, com `pandas.merge()`, e não por replicação ou referência cruzada entre os bancos — preservando a separação de responsabilidades entre o motor relacional e o motor geoespacial.
- As conexões com SQLite e MongoDB são armazenadas em cache (`st.cache_resource`), evitando reabri-las a cada interação do usuário e mantendo o dashboard responsivo.

6. Tecnologias Utilizadas

Camada	Tecnologia
Interface / Dashboard	Streamlit
Banco Relacional	SQLite (módulo <code>sqlite3</code>)
Banco NoSQL / Geoespacial	MongoDB (<code>pymongo</code>)
Mapas interativos	Folium + <code>streamlit-folium</code>
Gráficos analíticos	Plotly Express
Manipulação de dados	Pandas

7. Conclusão

A plataforma GeoLog demonstra, na prática, como a Persistência Poliglota permite que cada motor de banco de dados seja empregado em seu ponto forte — o SQLite garantindo consistência relacional para os dados cadastrais, e o MongoDB oferecendo indexação geoespacial nativa para a telemetria de alta frequência — sem que a aplicação final perca coesão, já que a camada Streamlit é responsável por unificar essas fontes em uma experiência analítica única para a diretoria da LogiTech Express.